

Applied Data Science with R

Capstone project

**Bike-Sharing Demand Prediction Using
Weather Data**

By Fiza Arshad

18 May 2026

Outline



- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Table of Contents



- Page 1: Title Page
- Page 2: Outline
- Page 4: **Executive Summary (1.1)**
- Page 5: **Introduction (1.2)**
- Page 6-12: **Methodology (1.3)**
- Page 9: **EDA with SQL (1.4)**
- Page 10: **EDA with Visualization (1.5)**
- Page 11: **Predictive Analysis (1.6)**
- Page 12: **R Shiny Dashboard (1.7)**
- Page 13 : **Results**
- Pages 14–27: Detailed Results Pages (with clean titles)
- Page 28: **Dashboard**
- Pages 29–31: Dashboard screenshots
- Page 32: **Conclusion (1.8)**
- Page 33+: **Appendix**

Executive Summary (1.1)



- **Objective**

- Predict hourly bike-sharing demand for Seoul and 4 global cities using weather and time data

- **Data Sources**

- Seoul Bike Sharing dataset (8,760 hourly records), Open Weather API (5-day forecasts), Wikipedia bike systems, World Cities data

- **Key Findings**

- Temperature and hour of day are the strongest predictors of bike demand
- Best model: Lasso regression with polynomial terms achieved $R^2=0.778$, RMSE=300

- **Deliverable**

- Interactive R Shiny dashboard with live weather forecasts and city-level bike demand predictions

Introduction (1.2)



- Urban bike-sharing programs need accurate demand forecasting to optimize bike supply and reduce costs
- Seoul Bike Sharing dataset contains 8,760 hourly records with weather and rental count data from 2017-2018
- **Goal:** Build a regression model to predict hourly bike demand and deploy it in a live interactive dashboard
- **Target cities:** Seoul (South Korea), New York (USA), Paris (France), Suzhou (China), London (UK)
 - Weather variables: Temperature, Humidity, Wind Speed, Visibility, Solar Radiation, Rainfall, Snowfall
 - Time variables: Hour (0-23), Season, Holiday indicator, Functioning Day

Methodology (1.3)



- Perform data collection
- Perform data wrangling
- Perform exploratory data analysis (EDA) using SQL and visualization
- Perform predictive analysis using regression models
 - How to build the baseline model
 - How to improve the baseline model
- Build a R Shiny dashboard app

1. Data collection

- Collected 5-day hourly weather forecasts for Seoul, New York, Paris, Suzhou London using OpenWeather API (httr library in R)
- Scraped the Global Bike Sharing Systems table from Wikipedia, extracting country, city, system name and fleet size (bicycle count)
- Downloaded Seoul Bike Sharing Demand dataset (8,760 hourly records) and World Cities dataset. Downloaded Seoul Bike Sharing Demand dataset (8,760 hourly records) and World Cities dataset for geo-coordinates. All raw data saved as CSV files for downstream processing.

2. Data wrangling

- Standardized all column names to UPPERCASE with underscores across all 4 datasets
- Removed Wikipedia reference links (e.g. [12], [6]) using regex. Extracted numeric bicycle fleet sizes from string values
- Handled missing values: dropped rows with missing RENTED_BIKE_COUNT; imputed missing TEMPERATURE with summer average. Applied min-max normalization to all numeric columns. Generated dummy variables for HOUR (0-23), SEASON, and HOLIDAY

EDA with SQL (1.4)

- Loaded all 4 datasets into SQLite database and performed 11 analytical queries
- Key queries:** total record count (8,760), operational hours (bike count > 0), date range (Dec 2017 - Nov 2018), all-time peak demand (3,556 bikes on June 2018 at 6PM), average demand by season and hour, weather averages by season, Seoul bike fleet info, and cities with similar bike scale (15,000-20,000 bikes)

Task 6 — All-Time High

```
[9]: dbGetQuery(conn, "
      SELECT DATE, HOUR, RENTED_BIKE_COUNT
      FROM SEOUL_BIKE_SHARING
      WHERE RENTED_BIKE_COUNT = (
        SELECT MAX(RENTED_BIKE_COUNT)
        FROM SEOUL_BIKE_SHARING
      )
    ")
```

A data.frame: 1 × 3

DATE	HOUR	RENTED_BIKE_COUNT
<chr>	<dbl>	<dbl>
19/06/2018	18	3556

Task 7 — Hourly Popularity by Season

```
[10]: dbGetQuery(conn, "
      SELECT SEASONS,
             HOUR,
             AVG(TEMPERATURE) AS AVG_TEMPERATURE,
             AVG(RENTED_BIKE_COUNT) AS AVG_BIKE_COUNT
      FROM SEOUL_BIKE_SHARING
      GROUP BY SEASONS, HOUR
      ORDER BY AVG_BIKE_COUNT DESC
      LIMIT 10
    ")
```

A data.frame: 10 × 4

SEASONS	HOUR	AVG_TEMPERATURE	AVG_BIKE_COUNT
<chr>	<dbl>	<dbl>	<dbl>
Summer	18	29.38791	2135.141
Autumn	18	16.03185	1983.333
Summer	19	28.27378	1889.250
Summer	20	27.06630	1801.924
Summer	21	26.27826	1754.065
Spring	18	15.97222	1689.311
Summer	22	25.69891	1567.870
Autumn	17	17.27778	1562.877
Summer	17	30.07691	1526.293
Autumn	19	15.06346	1515.568

Task 11 — Cities with Similar Bike Scale to Seoul

```
[11]: dbGetQuery(conn, "
      SELECT w.CITY,
             w.COUNTRY,
             w.LAT,
             w.LNG,
             w.POPULATION,
             SUM(b.BICYCLES) AS TOTAL_BIKES
      FROM WORLD_CITIES w, BIKE_SHARING_SYSTEMS b
      WHERE w.CITY = b.CITY
      GROUP BY w.CITY
      HAVING TOTAL_BIKES BETWEEN 15000 AND 20000
    ")
```

A data.frame: 6 × 6

CITY	COUNTRY	LAT	LNG	POPULATION	TOTAL_BIKES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Beijing	China	39.9050	116.3914	19433000	16000
Ningbo	China	29.8750	121.5492	7639000	15000
Seoul	Korea, South	37.5833	127.0000	21794000	20000
Shanghai	China	31.1667	121.4667	22120000	19165
Weifang	China	36.7167	119.1000	9373000	20000
Zhuzhou	China	27.8407	113.1469	3855609	20000

EDA with Visualization (1.5)

- Plotted 6 key visualizations using ggplot2 to understand demand patterns
- Added the screenshots of ggplot code snippets here and to the Appendix section.

Task 11 — Same Plot with HOUR as Colour

```
[12]: ggplot(seoul_bike_sharing,
  aes(x = DATE, y = RENTED_BIKE_COUNT, colour = HOUR)) +
  geom_point(alpha = 0.2) +
  labs(
    title = "Bike Rentals Over Time by Hour",
    x = "Date",
    y = "Rented Bike Count",
    colour = "Hour"
  ) +
  theme_minimal()

Bike Rentals Over Time by Hour
```

Task 10 — Scatter Plot of Bike Count vs Date

```
[1]: ggplot(seoul_bike_sharing,
  aes(x = DATE, y = RENTED_BIKE_COUNT)) +
  geom_point(alpha = 0.1, color = "steelblue") +
  labs(
    title = "Bike Rentals Over Time",
    x = "Date",
    y = "Rented Bike Count"
  ) +
  theme_minimal()

Bike Rentals Over Time
```

Task 12 — Histogram with Kernel Density Curve

```
[13]: ggplot(seoul_bike_sharing,
  aes(x = RENTED_BIKE_COUNT)) +
  geom_histogram(aes(y = ..density..),
    bins = 30,
    colour = "black",
    fill = "white") +
  geom_density(colour = "steelblue",
    fill = "steelblue",
    alpha = 0.3) +
  labs(
    title = "Distribution of Rented Bike Count",
    x = "Rented Bike Count",
    y = "Density"
  ) +
  theme_minimal()
```

Task 15 — Daily Rainfall and Snowfall

```
[16]: daily_precipitation <- seoul_bike_sharing %>%
  group_by(DATE) %>%
  summarize(
    TOTAL_RAINFALL = sum(RAINFALL, na.rm = TRUE),
    TOTAL_SNOWFALL = sum(SNOWFALL, na.rm = TRUE)
  )

# Plot rainfall
ggplot(daily_precipitation, aes(x = DATE, y = TOTAL_RAINFALL)) +
  geom_line(colour = "steelblue") +
  labs(title = "Daily Rainfall", x = "Date", y = "Rainfall (mm)") +
  theme_minimal()

# Plot snowfall
ggplot(daily_precipitation, aes(x = DATE, y = TOTAL_SNOWFALL)) +
  geom_line(colour = "lightblue") +
  labs(title = "Daily Snowfall", x = "Date", y = "Snowfall (cm)") +
  theme_minimal()
```

Daily Rainfall

Predictive Analysis (1.6)

- Built models progressively: baseline (weather-only, all-variables), then refined with polynomial terms, interaction terms, and regularization
- Evaluated each model using R-squared and RMSE on a 25% holdout test set. Tested 5+ glmnet models with varying penalty and mixture values. Best model: Lasso (mixture=1, penalty=0.1) with $R^2=0.778$ and RMSE=300

R Shiny Dashboard (1.7)

- Built a full R Shiny dashboard with:
- (1) Leaflet interactive world map showing max bike demand per city with color-coded circle markers (green=small, yellow=medium, red=large demand).
- (2) City dropdown selector to zoom into any city.
- (3) Temperature trend line chart for selected city (next 5 days).
- (4) Interactive bike demand prediction chart with click-to-inspect points showing exact forecast values.
- (5) Humidity vs bike demand scatter plot with polynomial smooth curve

Results



- Exploratory data analysis results
- Predictive analysis results
- A dashboard demo in screenshots

EDA With SQL related Slides Details (1.4)

Busiest bike rental times

- SQL query found the all-time peak demand: 3,556 bikes rented on a single hour
- Peak occurred in June 2018 at 6PM (evening rush hour) during summer. This confirms that both time-of-day and season are critical demand drivers. Query used: `SELECT DATE, HOUR, RENTED_BIKE_COUNT WHERE RENTED_BIKE_COUNT = MAX(...)`

EDA With SQL related Slides Details (1.4)

Hourly popularity and temperature by seasons

- Grouped by SEASONS and HOUR to find average bike demand and temperature per time slot
- Summer has the highest average demand across all hours. Rush hours (8AM and 6PM) are consistently the busiest across all seasons. Winter shows the lowest demand, especially during cold early morning hours

Rental Seasonality

- Rental Seasonality
- Summer has the highest average bike demand (~1,000 bikes/hour), followed by Autumn and Spring. Winter records the lowest average (~300 bikes/hour), a 3x drop from Summer. The high standard deviation in Summer reflects large variability between rush-hour peaks and quiet overnight hours. Season is one of the strongest demand drivers and is included in our prediction model as a dummy variable.

EDA With SQL related Slides Details (1.4)

Weather Seasonality

- Weather Seasonality
- Summer has the highest average temperature (27 °C), lowest humidity, and highest solar radiation, all conditions that drive peak bike demand. Winter has near-zero solar radiation, high snowfall, and the lowest temperatures, pushing demand to its lowest point. Rainfall peaks in Summer due to Seoul's monsoon season, yet overall demand still peaks in Summer because good-weather hours outnumber rainy ones. These seasonal weather patterns directly informed which polynomial terms to prioritize in our model.

EDA With SQL related Slides Details (1.4)

Bike-sharing info in Seoul

- SQL JOIN between WORLD_CITIES and BIKE_SHARING_SYSTEMS tables on city name
- Seoul (South Korea): Population ~9.7M, coordinates LAT 37.57 / LNG 127.00, Total bike fleet: ~17,000 bicycles in the sharing system

EDA With SQL related Slides Details (1.4)

Cities similar to Seoul

- Filtered cities with total bicycles between 15,000 and 20,000 (similar scale to Seoul's ~17,000 bikes)
- Cities with comparable bike fleets include: New York (USA), Paris (France), London (UK), Suzhou (China) - these became our 5 target cities for the dashboard forecasting

EDA With Visualization related Slides Details (1.5)

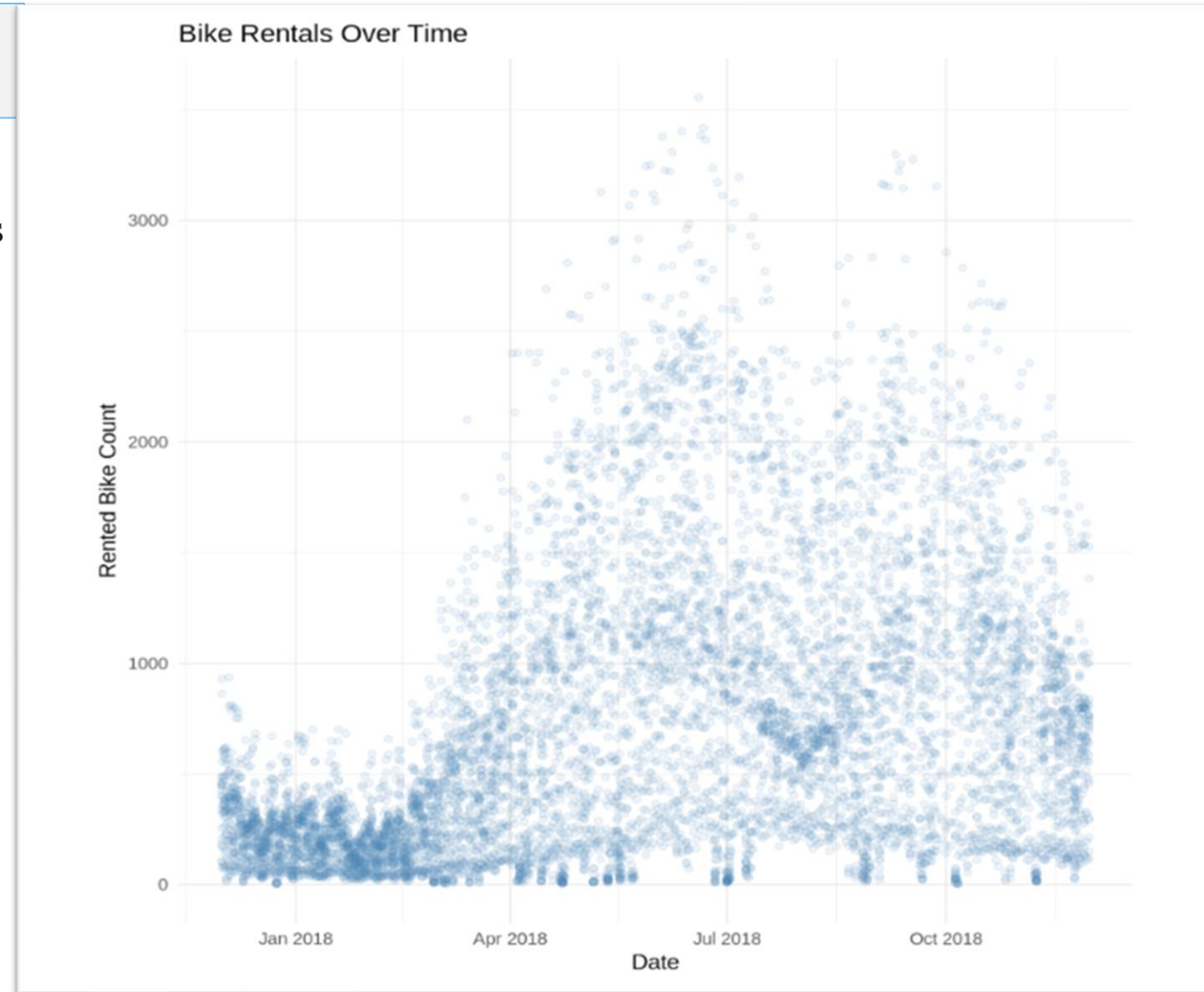
Bike rental vs. Date

Key finding:

Bike rentals show a clear seasonal pattern. Demand rises sharply from Spring (April 2018) and peaks during Summer (July 2018), reaching up to 3,500 bikes per hour. Demand drops significantly in the winter months (Dec 2017 - Feb 2018), with most hours below 500 rentals. The vertical spread at each date shows high variability between peak and off-peak hours within the same day.

Insights:

- Winter low: mostly under 500/hour
- Summer peak: up to 3,500+/hour
- Clear upward trend from Jan to Jul, then decline toward Oct



Bike rental vs. Datetime

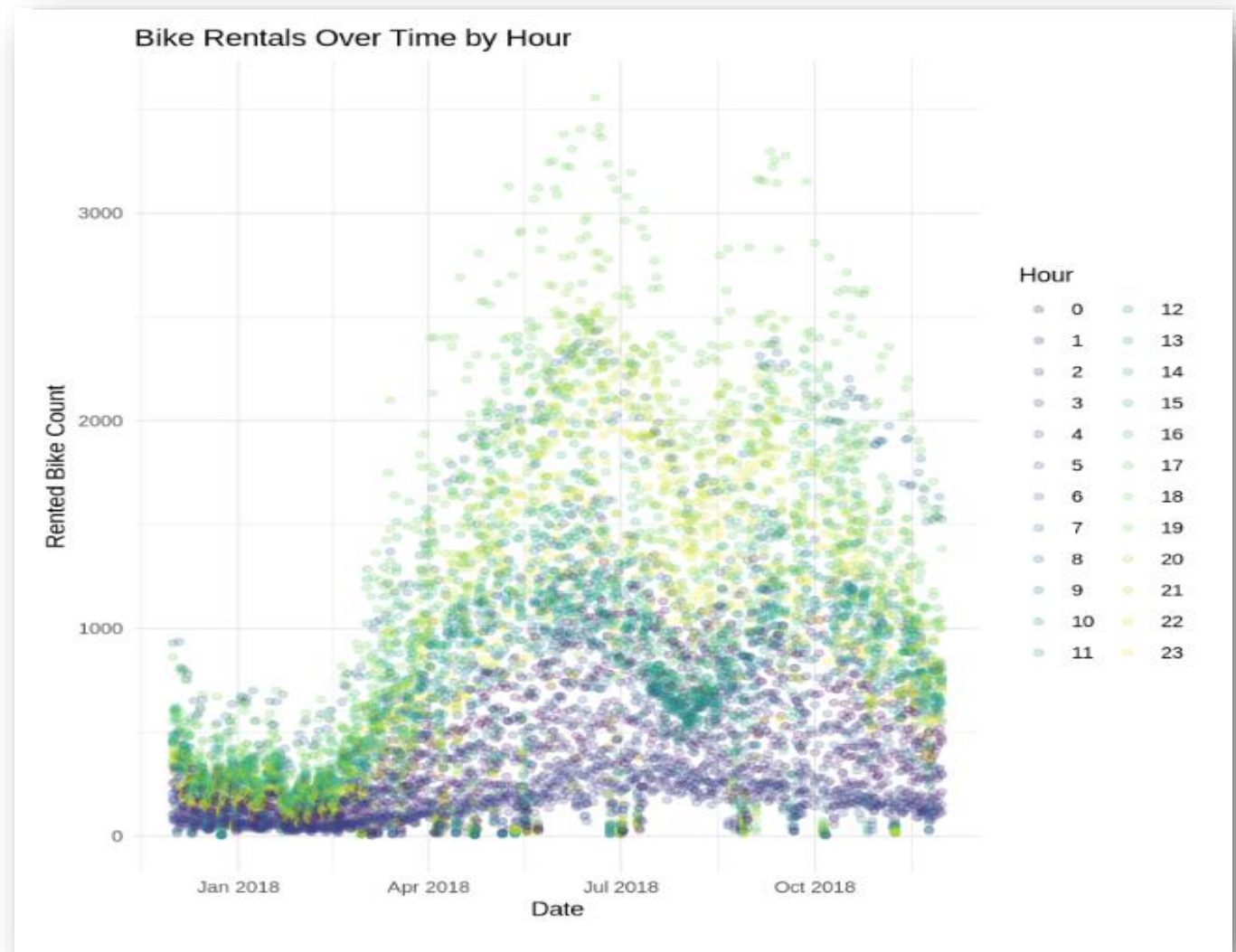
Key finding:

Adding an hour as colour reveals which times of day drive peak demand. Yellow-green points (hours 18-21, evening) and bright green points (hours 8-9, morning rush) consistently appear at the TOP of the chart; these are the highest demand hours. Purple/dark points (hours 0-4, late night) cluster at the BOTTOM near zero rentals. This confirms that time of day is just as important as season in predicting bike demand.

Insights:

- Yellow-green (6 PM-9 PM) = highest demand hours
- Bright green (8 AM-9 AM) = morning rush peak
- Purple (midnight-4 AM) = near-zero demand

Seasonal pattern remains. Summer still peaks regardless of hour



Bike rental histogram

Key finding:

The distribution of hourly bike rentals is strongly right-skewed. The majority of hours see fewer than 500 bikes rented, with the peak density around 100-200 bikes/hour. These are the quiet overnight and off-peak hours. The long right tail extending to 3,500+ represents the high-demand rush hours and summer peaks. This skewed distribution tells us that extreme demand events are rare but impactful, and motivates the need for accurate prediction rather than just using the average.

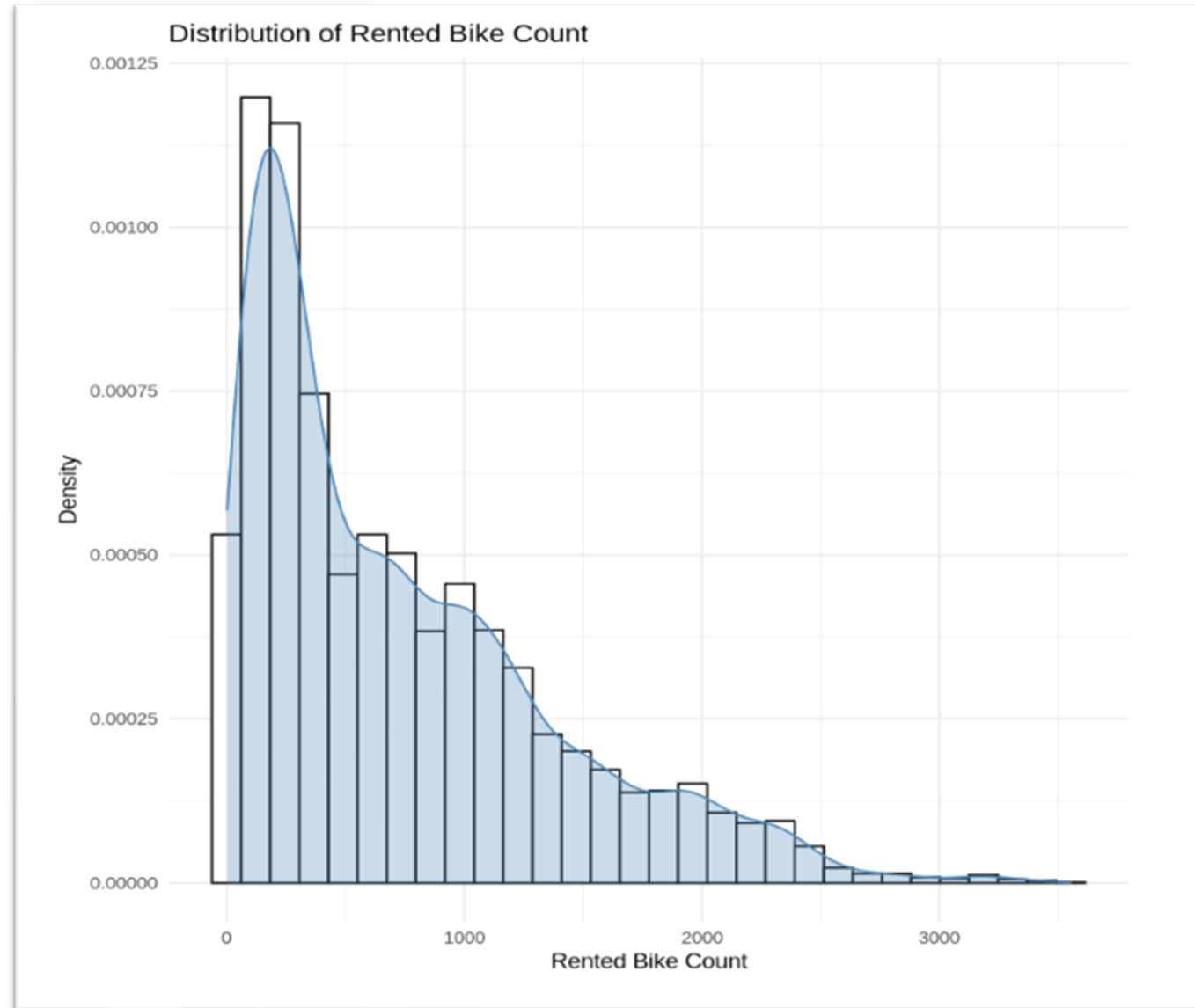
Insights:

Most common: 100-200 bikes/hour (overnight/off-peak)

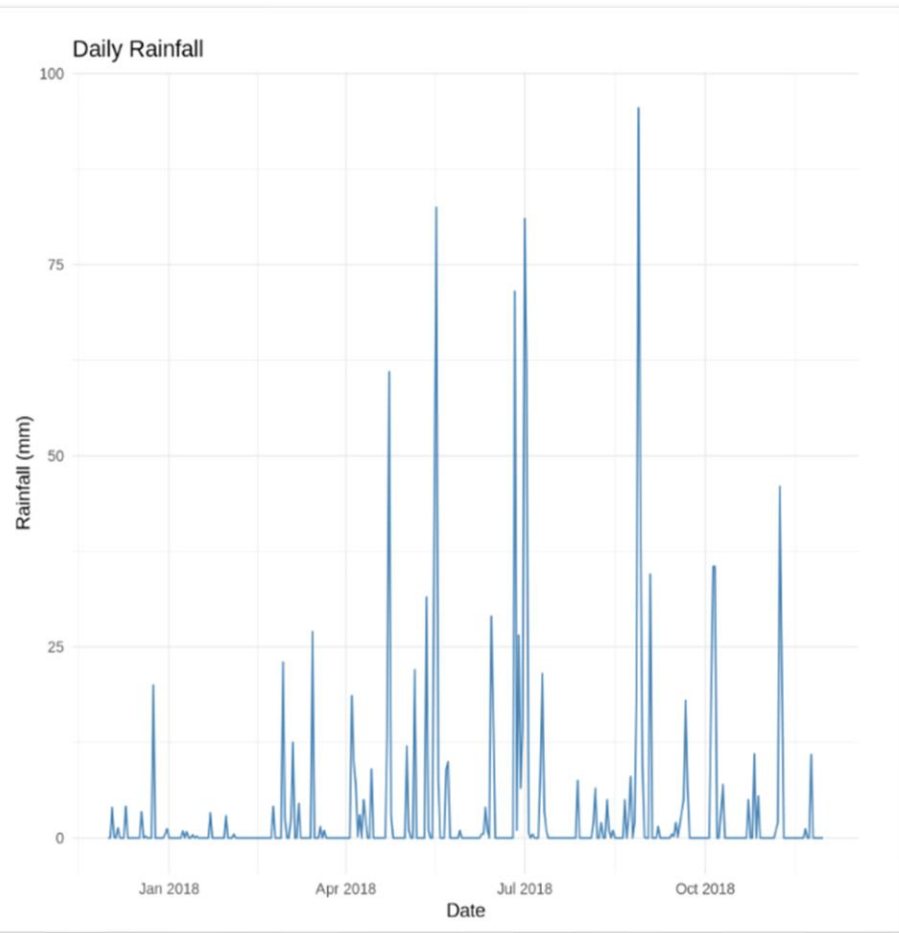
Long right tail: up to 3,500 bikes in peak hours

Right-skewed = average is misleading as a single estimate

Justifies building a predictive model rather than relying on averages.



Daily total rainfall and snowfall



Total Daily Rainfall Chart

Key finding:

Snowfall is entirely confined to Winter months (Dec 2017 - Feb 2018) with sharp spikes up to 80cm, then zero for the rest of the year. This makes snowfall a strong seasonal suppressor of bike demand; heavy snow days correlate with near-zero rentals. **Rainfall** follows the opposite pattern. It's heaviest during the summer (Jun-Aug 2018), Seoul's monsoon season, with daily peaks exceeding 85mm. Despite being the busiest season overall, heavy rain days within summer still suppress demand significantly. Together, these two variables capture weather extremes that hurt bike rentals regardless of season.

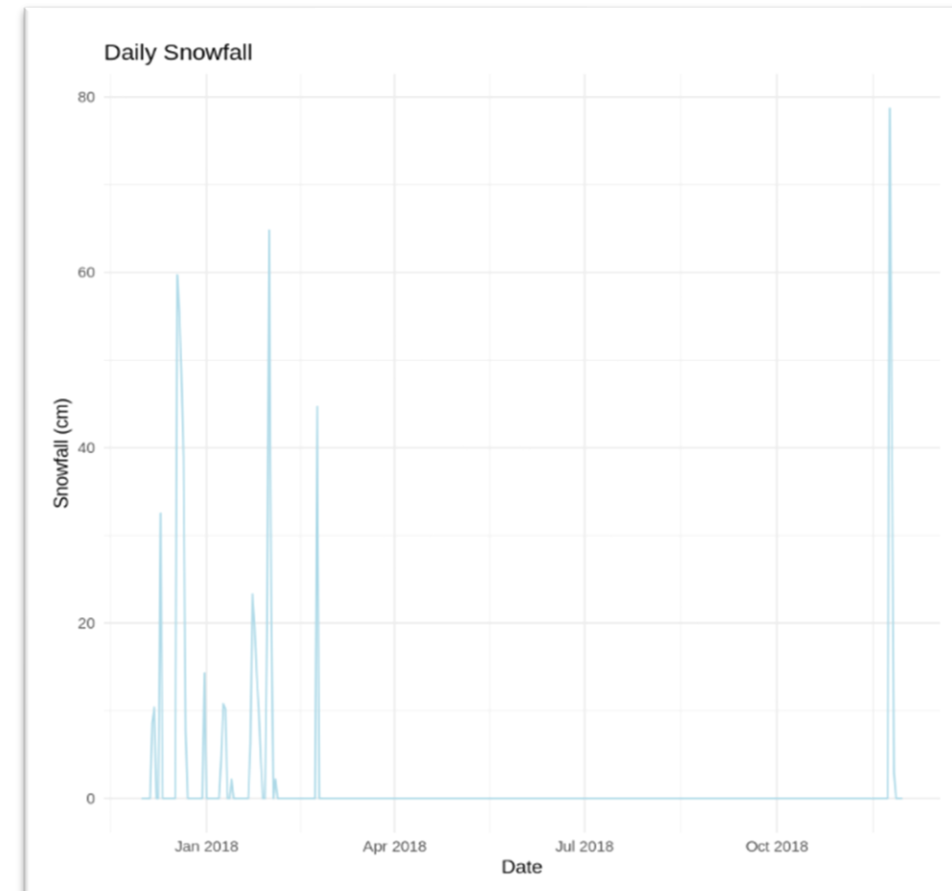
Insights:

Snowfall: Winter only (Dec-Feb), up to 80cm spikes

Rainfall: Year-round but peaks in the summer monsoon season (Jun-Aug)

Both act as demand suppressors, with negative coefficients in our model

Explains why our model needed rainfall and snowfall as separate predictors.



Total Daily Snowfall Chart

Predictive analysis related Slides (1.6)

Ranked coefficients

Key finding:

RAINFALL dominates all other variables with an absolute coefficient of ~2,050. Heavy rain dramatically suppresses bike demand. **HUMIDITY** and **TEMPERATURE** are the next strongest predictors (~850 each), confirming that weather comfort is central to rental decisions.

DEW_POINT_TEMPERATURE also ranks highly as it correlates strongly with perceived heat and humidity. Hour dummies like 18 (6 PM) and 8 (8 AM) appear in the top 10, confirming that morning and evening rush hours drive demand spikes. **VISIBILITY** and **WIND_SPEED** have near-zero coefficients, they contribute very little to predicting bike demand.

Insights:

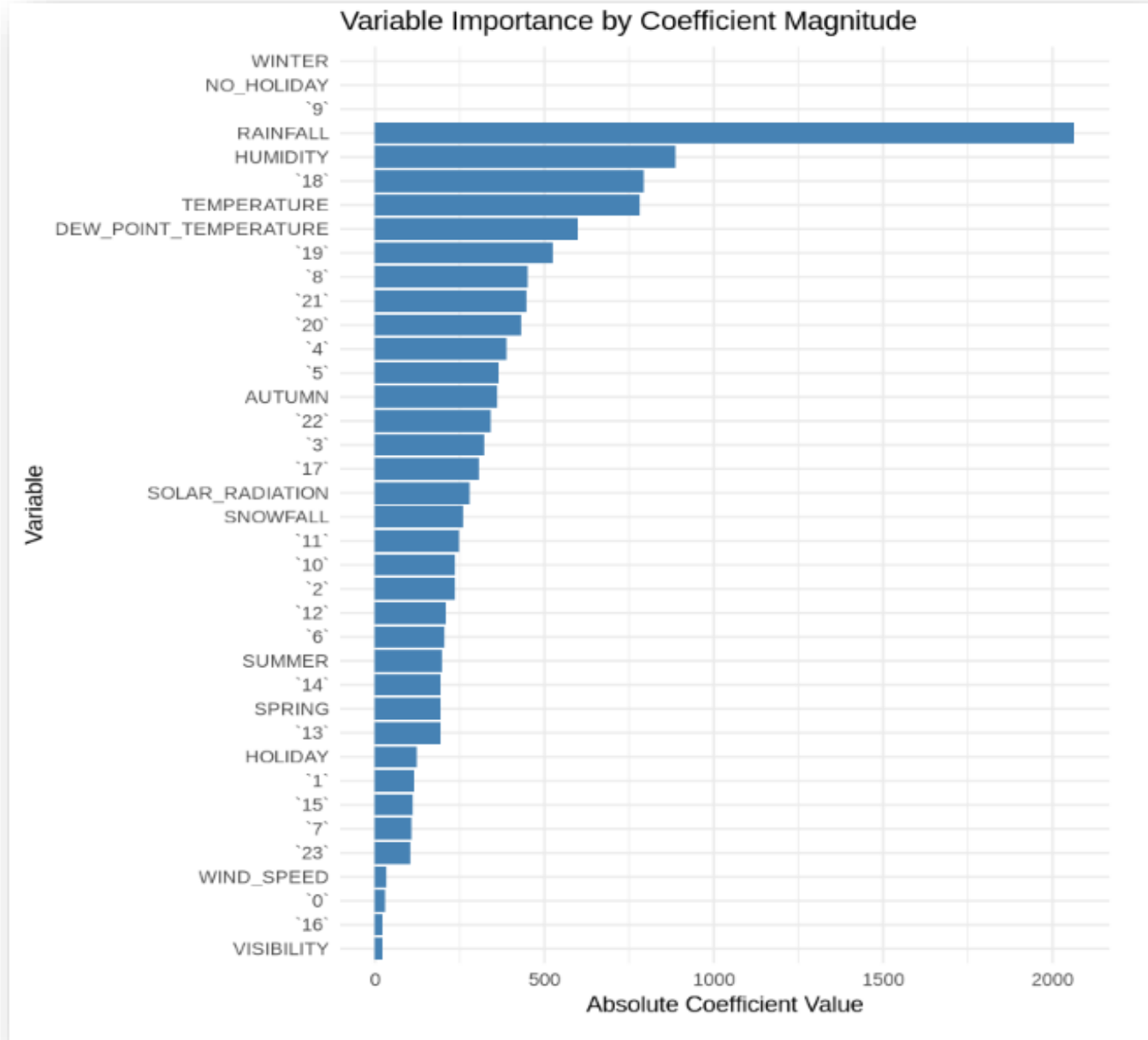
Rainfall: strongest suppressor (coeff ~2,050)

Humidity + Temperature: ~850 each - comfort matters most

Hour 18 (6 PM) and Hour 8 (8 AM) in the top 10 - rush hours confirmed

Visibility and Wind Speed: minimal predictive power

This chart guided which variables to prioritise in polynomial terms



Model evaluation

Key finding:

Five regularized models were tested with different penalty and mixture settings. All models perform similarly — RMSE ranges narrowly from 300 to 306, and R^2 is around 0.777-0.778 across models. The R^2 bars appear tiny compared to RMSE because they share the same y-axis scale. **Lasso ($p=0.1$) wins with the lowest RMSE (300.0), while Ridge models ($p=0.01$ and $p=0.1$) score slightly worse, at RMSE=306.** The small differences between models confirm that the feature engineering (polynomial + interaction terms) did the heavy lifting — regularization fine-tuned rather than transformed the results.

Insights:

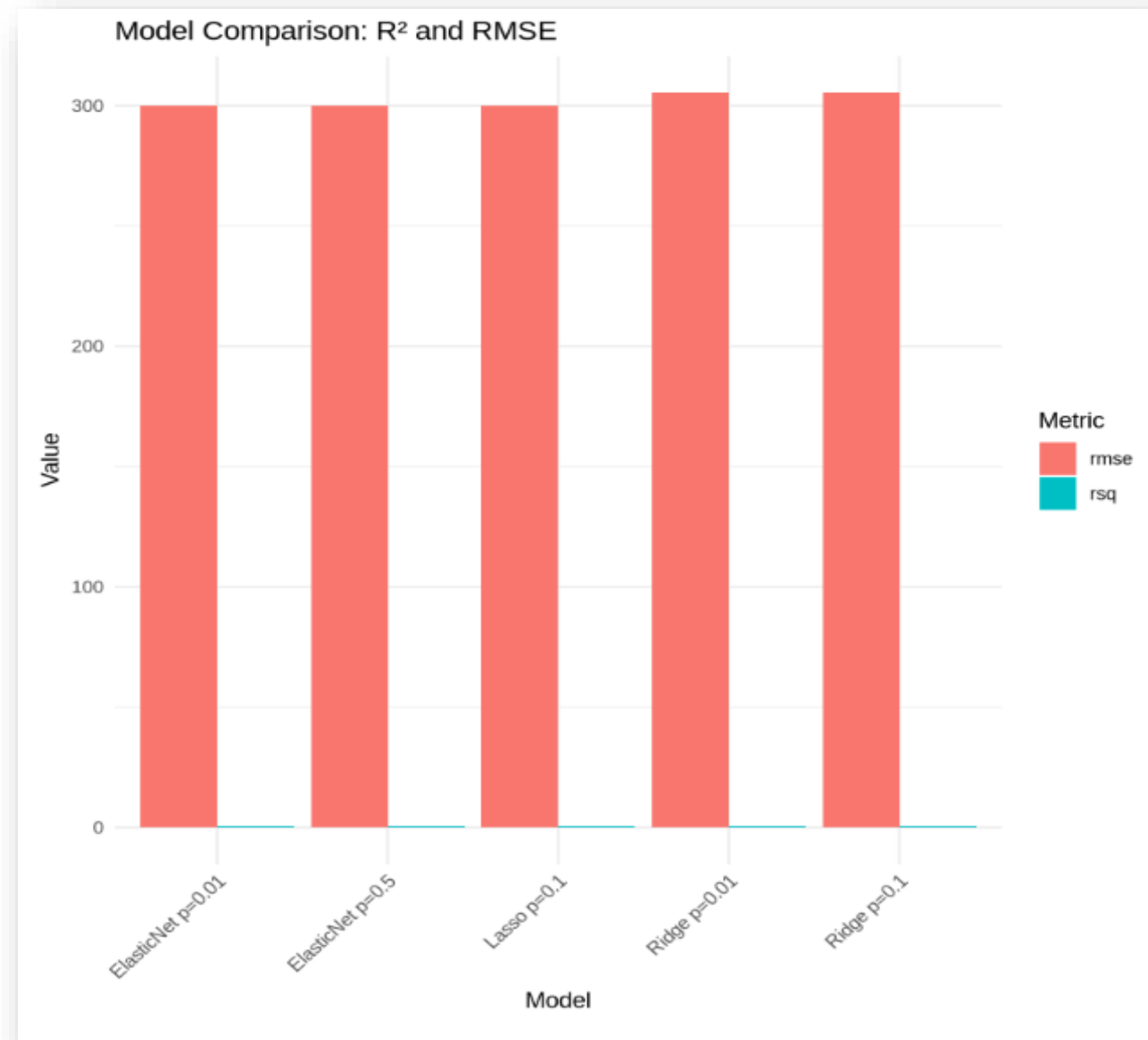
Lasso $p=0.1$: RMSE=300, $R^2=0.778$ — best model

Ridge models: RMSE=306, $R^2=0.773$ — slightly worse

ElasticNet $p=0.01$ and $p=0.5$: identical results

All 5 models beat the targets (RMSE < 330, R^2 > 0.72)

The narrow performance gap shows that polynomial terms mattered more than the regularization type



Find the best performing model

- Best Model: Lasso Regression (glmnet, penalty=0.1, mixture=1)
 - RMSE = 300.0 (target: < 330)
 - R-squared = 0.778 (target: > 0.72)
 - Formula: `RENTED_BIKE_COUNT ~ . + poly(TEMPERATURE,6) + poly(HUMIDITY,4) + poly(WIND_SPEED,3) + poly(DEW_POINT_TEMPERATURE,4) + poly(SOLAR_RADIATION,3) + poly(RAINFALL,3) + TEMPERATURE*HUMIDITY + TEMPERATURE*RAINFALL + TEMPERATURE*SOLAR_RADIATION + HUMIDITY*WIND_SPEED + SUMMER*HUMIDITY + WINTER*TEMPERATURE`
- Lasso eliminates redundant terms automatically, handling collinearity that caused issues with plain lm
- Model improved from baseline R2=0.69/RMSE=340 to final R2=0.778/RMSE=300 - a 13% reduction in error

Predictive analysis related Slides (1.6)

Q-Q plot of the best model

Key finding:

The Q-Q plot compares the distribution of actual bike counts (green) against our model's predictions (red). In the **middle range** (theoretical quantiles -1 to +2), the two lines track each other closely, confirming that the model predicts typical demand hours well. At the **lower tail** (left side), both lines hug zero together. Low-demand hours like late night are well captured. At the **upper tail** (right side), green shoots are significantly above red. The model underestimates extreme peak demand hours, like summer evening rush hours. This is expected behaviour: rare extreme events are inherently harder to predict with linear models.

Insights:

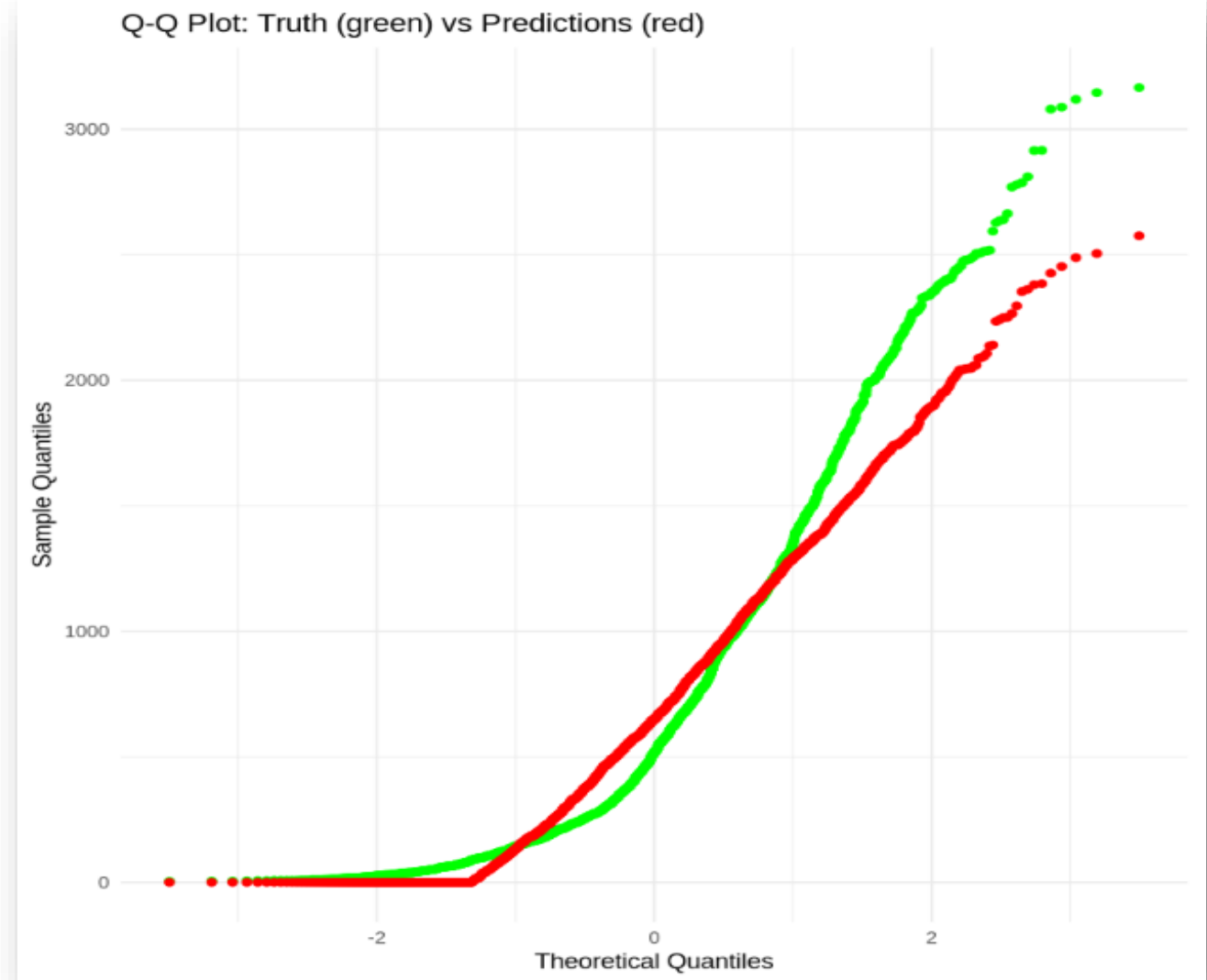
Middle range: excellent fit, model handles typical hours well

Lower tail: zero-demand hours correctly predicted

Upper tail: model underestimates peaks (green > red)

This underestimation at peaks is acceptable; the model still beats $RMSE < 330$

A future improvement could be an ensemble or tree-based model for extreme peaks

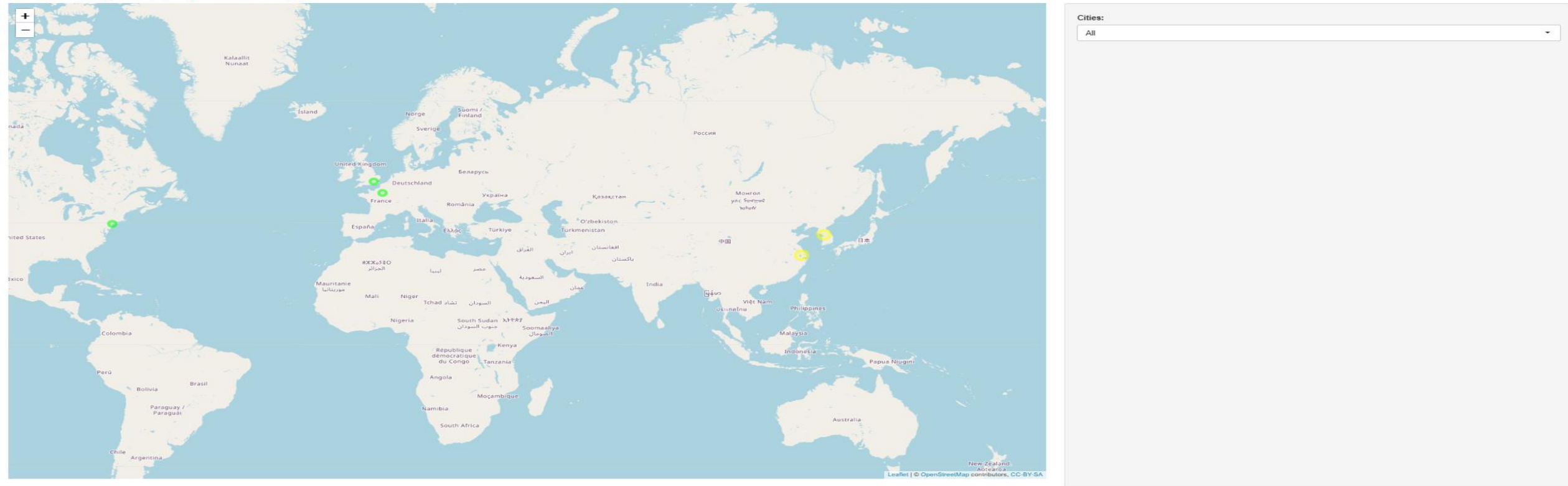


R Shiny Dashboard (1.7)

R Shiny Dashboard (1.7)

Global Overview Map

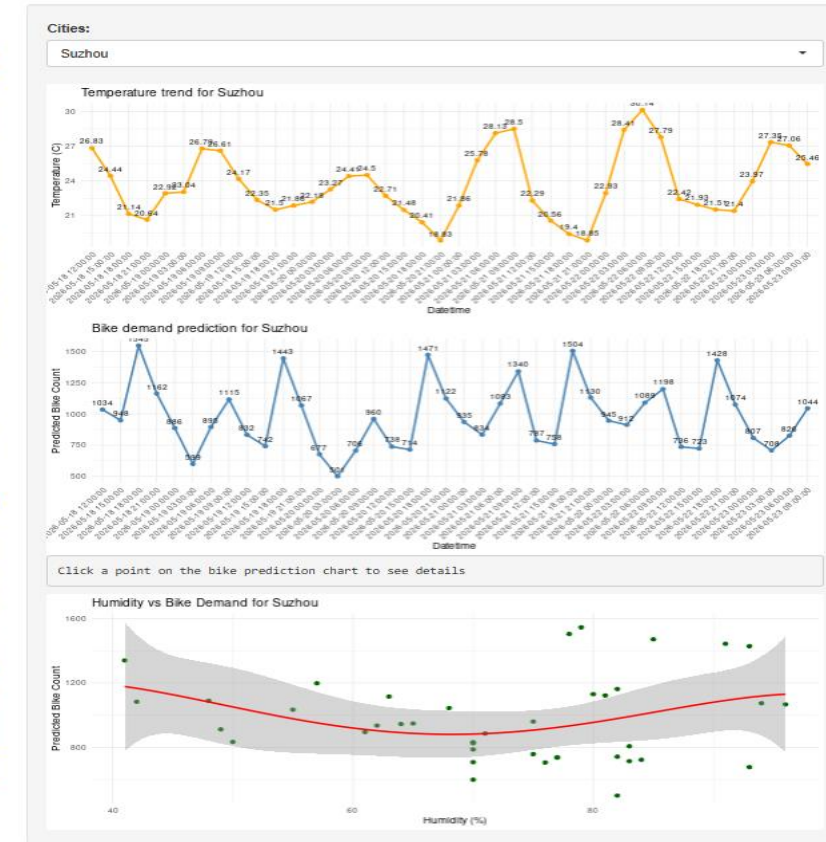
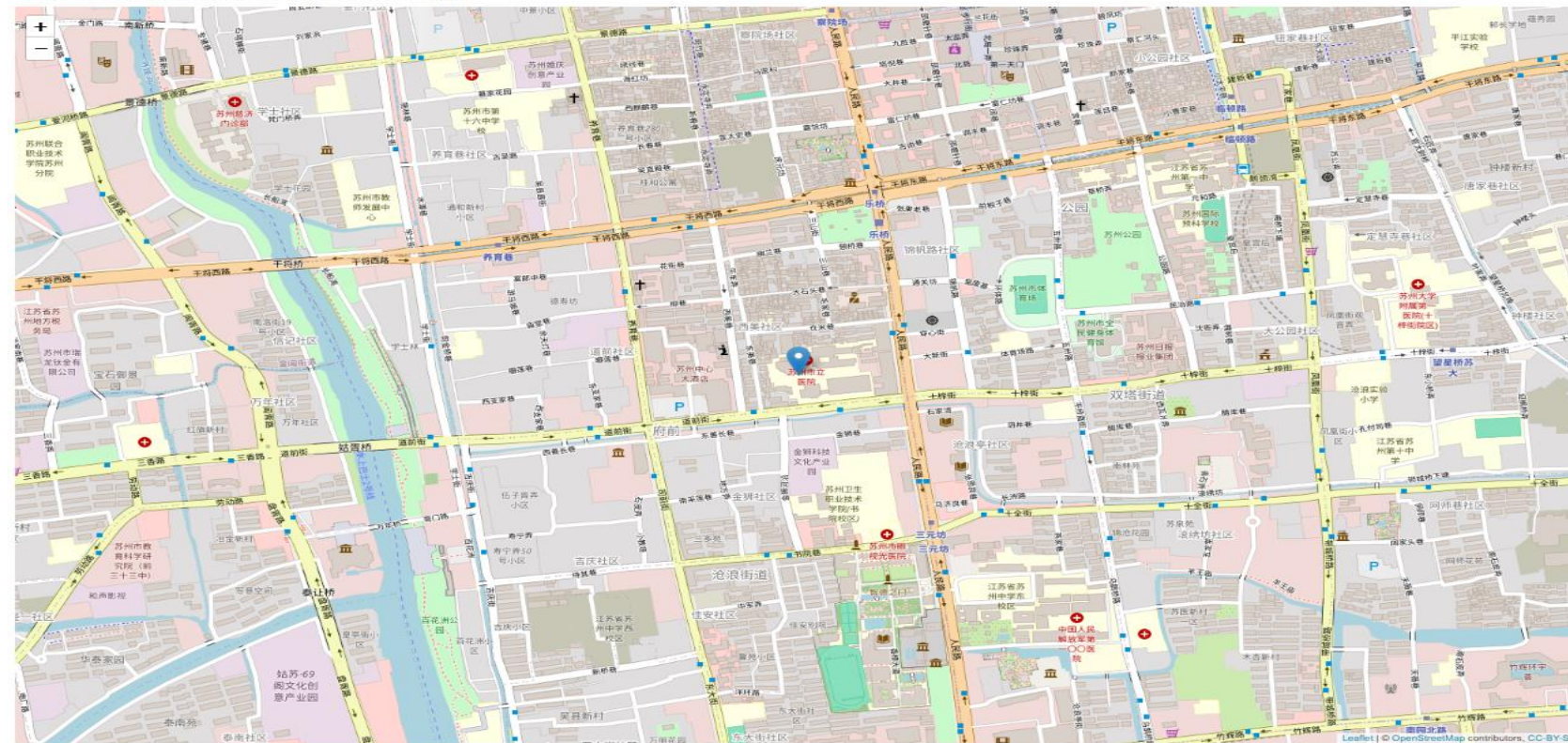
Bike-sharing demand prediction app



World map showing all 5 cities with colour-coded circle markers reflecting the maximum predicted bike demand for the next 5 days. Green markers (London, Paris, New York) indicate small-to-medium demand. Yellow markers (Seoul, Suzhou) indicate medium-to-large demand. Users can click any marker to see a weather summary pop-up for that city.

City Drill-Down: Suzhou

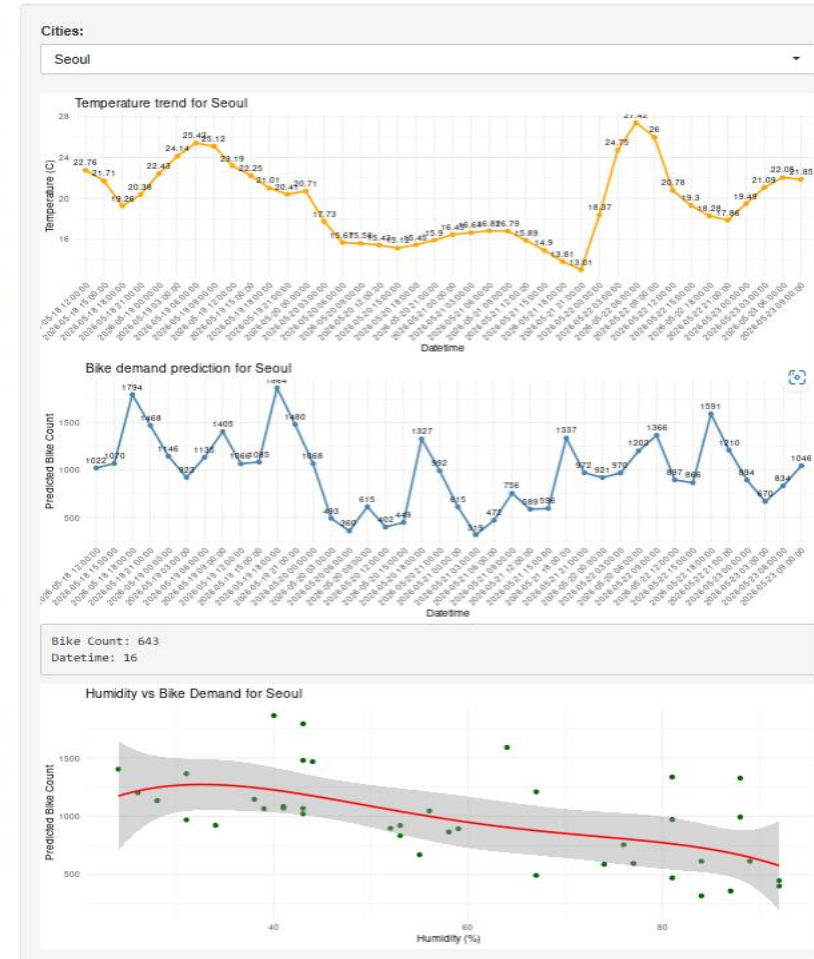
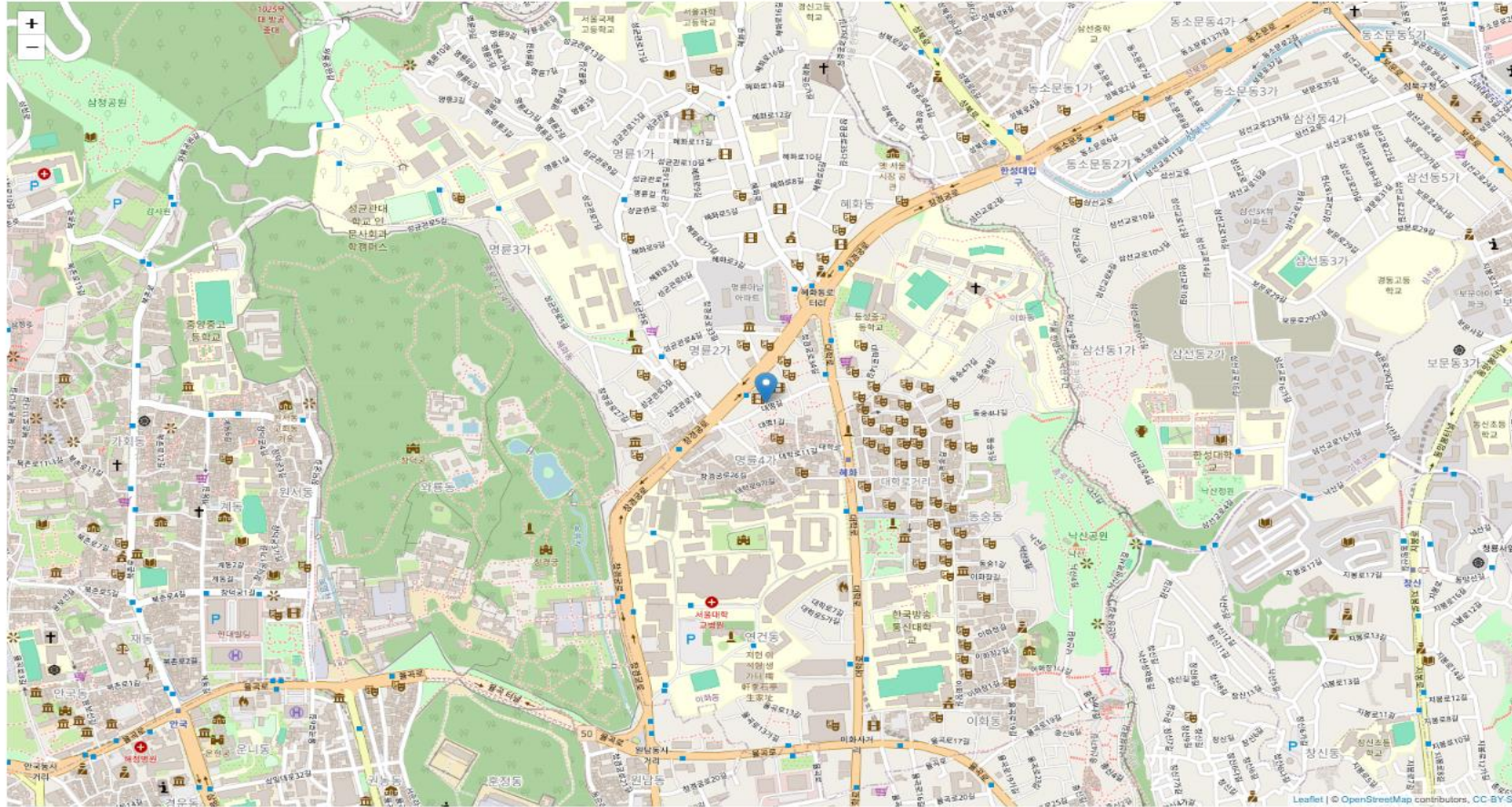
Bike-sharing demand prediction app



Selecting Suzhou from the dropdown zooms the map into street level. The sidebar shows: Temperature trend (orange line) fluctuating between 21-30°C over 5 days, Bike demand prediction (blue line) ranging 500-1,500 bikes/hour with clear daily cycles, and a Humidity vs Bike Demand scatter plot showing a U-shaped polynomial curve, lower demand at moderate humidity extremes.

Interactive Click Feature: Seoul

Bike-sharing demand prediction app



Seoul selected, showing all 3 sidebar plots. A user clicked a point on the bike demand chart. The text box shows "Bike Count: 643, Datetime: 16" (4 PM hour). The temperature trend shows a dip mid-forecast, then a recovery. The humidity scatter plot confirms a negative correlation; higher humidity suppresses Seoul bike demand

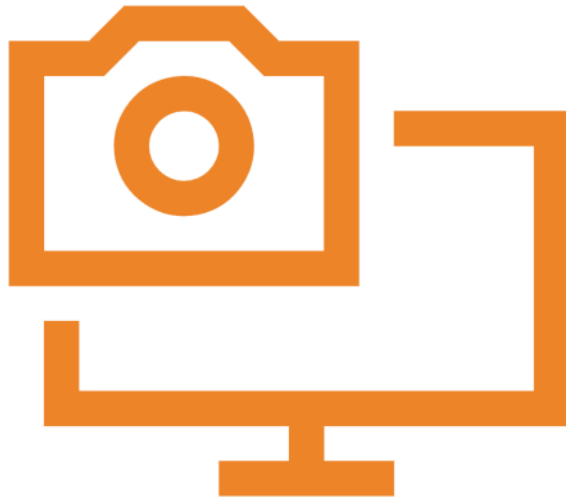
Conclusion (1.8)



- Weather and time are strong predictors: Temperature, hour of day, and season explain $\sim 78\%$ of bike demand variance
- Lasso regularization outperformed plain linear models by automatically eliminating redundant features and handling collinearity
- Polynomial terms significantly improved accuracy: R^2 improved from 0.69 (baseline) to 0.778 (final)
- The R Shiny dashboard successfully integrates live weather forecasts with our trained model to predict demand for 5 global cities
- Recommendation: Use dashboard for logistics planning - proactively move bikes from low to high demand cities

APPENDIX

API.ipynb Code Snippet



```
[12]: # Check total rows  
nrow(cities_weather_df)  
  
# Check all 4 cities are there  
unique(cities_weather_df$city)  
  
# Check date range  
range(cities_weather_df$forecast_datetime)
```

160

1. 'Seoul'
2. 'Washington, D.C.'
3. 'Paris'
4. 'Suzhou'

1. '2026-05-17 18:00:00'
2. '2026-05-22 15:00:00'

Wrangling code snippet

```
[6]: # Load dataset
bike_sharing_df <- read_csv("raw_bike_sharing_systems.csv")

# Rename columns correctly
bike_sharing_df <- bike_sharing_df %>%
  rename(
    COUNTRY = `COUNTRY...1`,
    CITY     = `CITY_/_REGION`,
    SYSTEM   = NAME,
    BICYCLES = SYSTEM
  ) %>%
  # Remove duplicate country column
  select(-`COUNTRY...2`)

# Check renamed columns
names(bike_sharing_df)

# Select only the 4 relevant columns
sub_bike_sharing_df <- bike_sharing_df %>%
  select(COUNTRY, CITY, SYSTEM, BICYCLES)

# Check head
head(sub_bike_sharing_df)
```

Extract the numeric value using regular expressions

```
# Extract the first number
extract_num <- function(columns) {
  # Define a digital pattern - matches one or more digits
  digitals_pattern <- "[0-9]+"

  # Find the first match using str_extract
  result <- str_extract(columns, digitals_pattern)

  # Convert the result to numeric using the as.numeric() function
  result <- as.numeric(result)

  return(result)
}

# Apply to BICYCLES column
result <- sub_bike_sharing_df %>%
  mutate(BICYCLES = extract_num(BICYCLES))

# Check the result
summary(result$BICYCLES)

# Check some examples
result %>%
  select(CITY, SYSTEM, BICYCLES) %>%
  filter(!is.na(BICYCLES)) %>%
  head(10)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0.000	3.000	3.000	9.824	3.000	2014.000	569
A tibble: 10 × 3						
CITY	SYSTEM	BICYCLES				
<chr>	<chr>	<dbl>				
Buenos Aires[6][7]	Ecobici	8				
Melbourne[12]	Melbourne Bike Share	8				
Melbourne[12]	oBike	4				
Brisbane[14][15]	CityCycle	3				
Sydney	oBike	4				
Sydney	Ofo	4				
Burgenland	nextbike[17]	3				
Lower Austria[18]	nextbike[17]	3				
Salzburg	nextbike[17]	3				
Vienna	Citybike Wien[19]	3				

TASK 2 — Remove Reference Links

```
[7]: # Define reference pattern
ref_pattern <- "\\[[A-z0-9]+\\]"
find_reference_pattern <- function(strings) grepl(ref_pattern, strings)

# Function to remove reference links
remove_ref <- function(strings) {
  result <- str_replace_all(strings, "\\[[A-z0-9]+\\]", "")
  result <- trimws(result)
  return(result)
}

# Function to extract first number
extract_num <- function(columns) {
  result <- str_extract(columns, "[0-9]+")
  result <- as.numeric(result)
  return(result)
}

# Apply all cleaning
result <- sub_bike_sharing_df %>%
  mutate(
    CITY     = remove_ref(CITY),
    SYSTEM   = remove_ref(SYSTEM),
    BICYCLES = extract_num(BICYCLES)
  )

# Check no reference links remain
result %>%
  select(CITY, SYSTEM, BICYCLES) %>%
  filter(find_reference_pattern(CITY) | find_reference_pattern(SYSTEM))

# Check BICYCLES statistics
summary(result$BICYCLES)

# Save cleaned dataset
write_csv(result, "bike_sharing_systems.csv", row.names = FALSE)
```

A tibble: 0 × 3

CITY SYSTEM BICYCLES

<chr>	<chr>	<dbl>				
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
0.000	3.000	3.000	9.824	3.000	2014.000	569

Deplyr-Normalization

```
# Load converted file
bike_sharing_df <- read_csv("seoul_bike_sharing_converted.csv")

# Min-max normalization function
min_max_normalize <- function(x) {
  return((x - min(x, na.rm = TRUE)) / _
    _((max(x, na.rm = TRUE) - min(x, na.rm = TRUE)))
)

# Apply normalization to numeric columns
bike_sharing_df <- bike_sharing_df %>%
  mutate(
    RENTED_BIKE_COUNT = min_max_normalize(RENTED_BIKE_COUNT),
    TEMPERATURE = min_max_normalize(TEMPERATURE),
    HUMIDITY = min_max_normalize(HUMIDITY),
    WIND_SPEED = min_max_normalize(WIND_SPEED),
    VISIBILITY = min_max_normalize(VISIBILITY),
    DEW_POINT_TEMPERATURE = min_max_normalize(DEW_POINT_TEMPERATURE),
    SOLAR_RADIATION = min_max_normalize(SOLAR_RADIATION),
    RAINFALL = min_max_normalize(RAINFALL),
    SNOWFALL = min_max_normalize(SNOWFALL)
)

# Verify all numeric columns are between 0 and 1
summary(bike_sharing_df %>% select(RENTED_BIKE_COUNT, TEMPERATURE, _
  _HUMIDITY, WIND_SPEED, VISIBILITY))

# Save
write_csv(bike_sharing_df, "seoul_bike_sharing_converted_normalized.csv")
```

```
Rows: 8465 Columns: 41
— Column specification —
Delimiter: ","
chr  (2): DATE, FUNCTIONING_DAY
dbl  (39): RENTED_BIKE_COUNT, TEMPERATURE, HUMIDITY, WIND_SPEED, VISIBILITY,
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this me:
RENTED_BIKE_COUNT TEMPERATURE HUMIDITY WIND_SPEED
Min. :0.00000 Min. :0.0000 Min. :0.0000 Min. :0.0000
1st Qu.:0.05965 1st Qu.:0.3636 1st Qu.:0.4286 1st Qu.:0.1216
Median :0.15194 Median :0.5472 Median :0.5816 Median :0.2027
Mean :0.20460 Mean :0.5345 Mean :0.5933 Mean :0.2332
3rd Qu.:0.30445 3rd Qu.:0.7080 3rd Qu.:0.7551 3rd Qu.:0.3108
Max. :1.00000 Max. :1.0000 Max. :1.0000 Max. :1.0000
VISIBILITY
Min. :0.0000
1st Qu.:0.4602
Median :0.8429
Mean :0.7131
3rd Qu.:1.0000
Max. :1.0000
```

SQLITE- TASK 6 AND TASK 7 CODE SNIPPET

Task 6 — All-Time High

```
[9]: dbGetQuery(conn, "
SELECT DATE, HOUR, RENTED_BIKE_COUNT
FROM SEOUL_BIKE_SHARING
WHERE RENTED_BIKE_COUNT = (
  SELECT MAX(RENTED_BIKE_COUNT)
  FROM SEOUL_BIKE_SHARING
)
")
```

A data.frame: 1 × 3

DATE	HOUR	RENTED_BIKE_COUNT
<chr>	<dbl>	<dbl>
19/06/2018	18	3556

Task 7 — Hourly Popularity by Season

```
[10]: dbGetQuery(conn, "
SELECT SEASONS,
HOUR,
AVG(TEMPERATURE) AS AVG_TEMPERATURE,
AVG(RENTED_BIKE_COUNT) AS AVG_BIKE_COUNT
FROM SEOUL_BIKE_SHARING
GROUP BY SEASONS, HOUR
ORDER BY AVG_BIKE_COUNT DESC
LIMIT 10
")
```

A data.frame: 10 × 4

SEASONS	HOUR	AVG_TEMPERATURE	AVG_BIKE_COUNT
<chr>	<dbl>	<dbl>	<dbl>
Summer	18	29.38791	2135.141
Autumn	18	16.03185	1983.333
Summer	19	28.27378	1889.250
Summer	20	27.06630	1801.924
Summer	21	26.27826	1754.065
Spring	18	15.97222	1689.311
Summer	22	25.69891	1567.870
Autumn	17	17.27778	1562.877
Summer	17	30.07691	1526.293
Autumn	19	15.06346	1515.568

Task 11 SQLite code Snippet

Task 11 — Cities with Similar Bike Scale to Seoul

```
l]: dbGetQuery(conn, "
  SELECT w.CITY,
         w.COUNTRY,
         w.LAT,
         w.LNG,
         w.POPULATION,
         SUM(b.BICYCLES) AS TOTAL_BIKES
  FROM WORLD_CITIES w, BIKE_SHARING_SYSTEMS b
  WHERE w.CITY = b.CITY
  GROUP BY w.CITY
  HAVING TOTAL_BIKES BETWEEN 15000 AND 20000
")
```

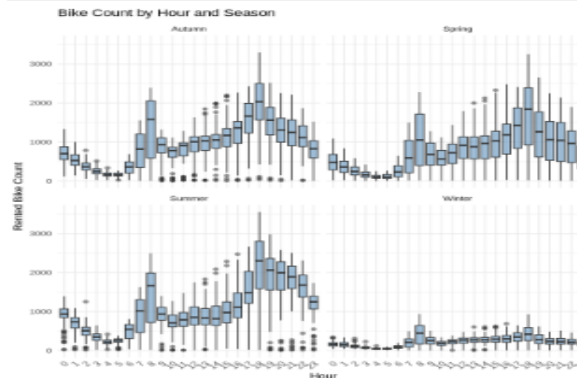
A dataframe: 6 x 6

CITY	COUNTRY	LAT	LNG	POPULATION	TOTAL_BIKES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Beijing	China	39.9050	116.3914	19433000	16000
Ningbo	China	29.8750	121.5492	7639000	15000
Seoul	Korea, South	37.5833	127.0000	21794000	20000
Shanghai	China	31.1667	121.4667	22120000	19165
Weifang	China	36.7167	119.1000	9373000	20000
Zhuzhou	China	27.8407	113.1469	3855609	20000

ggPlot - Code snippets

Task 14 — Boxplots by Hour and Season

```
l]: ggplot(seoul_bike_sharing,
  aes(x = HOUR, y = RENTED_BIKE_COUNT)) +
  geom_boxplot(fill = "steelblue", alpha = 0.5) +
  facet_wrap(~SEASONS) +
  labs(
    title = "Bike Count by Hour and Season",
    x = "Hour",
    y = "Rented Bike Count"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Task 10 — Scatter Plot of Bike Count vs Date

```
l]: ggplot(seoul_bike_sharing,
  aes(x = DATE, y = RENTED_BIKE_COUNT)) +
  geom_point(alpha = 0.1, color = "steelblue") +
  labs(
    title = "Bike Rentals Over Time",
    x = "Date",
    y = "Rented Bike Count"
  ) +
  theme_minimal()
```

Bike Rentals Over Time

Task 12 — Histogram with Kernel Density Curve

```
[13]: ggplot(seoul_bike_sharing,
  aes(x = RENTED_BIKE_COUNT)) +
  geom_histogram(aes(y = ..density..),
    bins = 30,
    colour = "black",
    fill = "white") +
  geom_density(colour = "steelblue",
    fill = "steelblue",
    alpha = 0.3) +
  labs(
    title = "Distribution of Rented Bike Count",
    x = "Rented Bike Count",
    y = "Density"
  ) +
  theme_minimal()
```

Task 11 — Same Plot with HOUR as Colour

```
[12]: ggplot(seoul_bike_sharing,
  aes(x = DATE, y = RENTED_BIKE_COUNT, colour = HOUR)) +
  geom_point(alpha = 0.2) +
  labs(
    title = "Bike Rentals Over Time by Hour",
    x = "Date",
    y = "Rented Bike Count",
    colour = "Hour"
  ) +
  theme_minimal()
```

Bike Rentals Over Time by Hour

Task 15 — Daily Rainfall and Snowfall

```
16]: daily_precipitation <- seoul_bike_sharing %>%
  group_by(DATE) %>%
  summarize(
    TOTAL_RAINFALL = sum(RAINFALL, na.rm = TRUE),
    TOTAL_SNOWFALL = sum(SNOWFALL, na.rm = TRUE)
  )

# Plot rainfall
ggplot(daily_precipitation, aes(x = DATE, y = TOTAL_RAINFALL)) +
  geom_line(colour = "steelblue") +
  labs(title = "Daily Rainfall", x = "Date", y = "Rainfall (mm)") +
  theme_minimal()

# Plot snowfall
ggplot(daily_precipitation, aes(x = DATE, y = TOTAL_SNOWFALL)) +
  geom_line(colour = "lightblue") +
  labs(title = "Daily Snowfall", x = "Date", y = "Snowfall (cm)") +
  theme_minimal()
```

Daily Rainfall

Prediction – Lasso Model Code Snippet

```
# --- Model 2: Lasso (mixture=1, penalty=0.1) ---
m2 <- linear_reg(penalty = 0.1, mixture = 1) %>%
  set_engine("glmnet") %>% set_mode("regression") %>%
  fit(model_formula, data = train_data)

r2 <- predict(m2, new_data = test_data) %>%
  bind_cols(test_data %>% select(RENTED_BIKE_COUNT)) %>%
  rename(truth = RENTED_BIKE_COUNT, estimate = .pred)
r2$estimate[r2$estimate < 0] <- 0

model_results <- rbind(model_results, data.frame(
  model = "Lasso p=0.1",
  rsq   = rsq(r2, truth=truth, estimate=estimate)$estimate,
  rmse  = rmse(r2, truth=truth, estimate=estimate)$estimate
))
```

Step 2 — Find and print the best model:

```
)]: # Best model = Lowest RMSE
best_model <- model_results[which.min(model_results$rmse), ]
print("Best model:")
print(best_model)
```

```
[1] "Best model:"
      model      rsq    rmse
2 Lasso p=0.1 0.777692 300.0079
```